

Exception Handling In Depth

Page No.:

Date: / /

Q What is Exception?

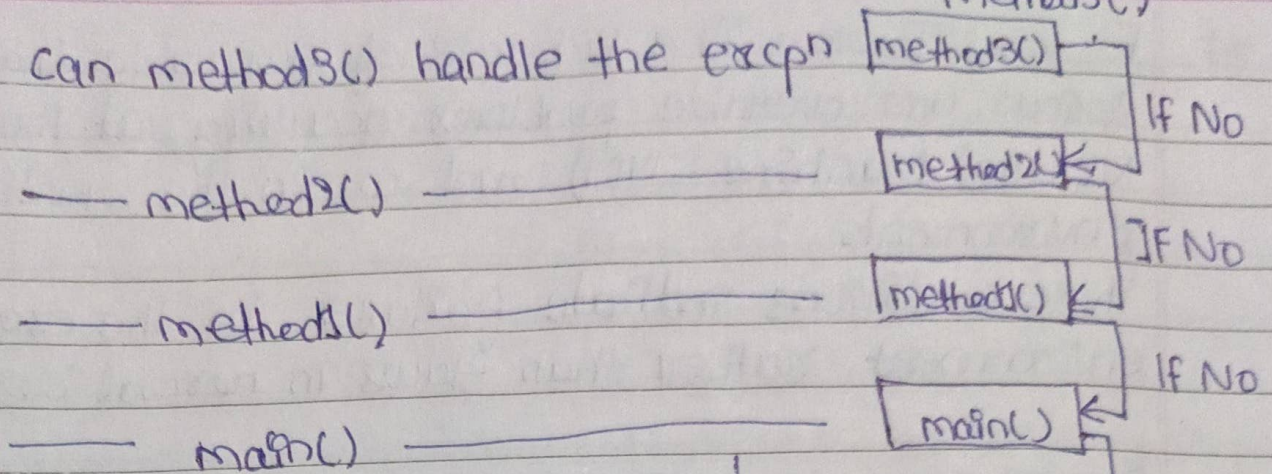
- It's an event that occurs during the execution of the program.
- It will disrupt your program normal flow.
- It creates the Exception Objects which contain information about the error like
 - type of Exception & message
 - ~~message~~ stack trace etc.
- Runtime system use this Exception Object and find the class which can handle it.

Ex:

```
public class Main {  
    psum() {  
        Main sampleObj = new Main();  
        sampleObj.method1();  
    }  
    private void method1() {  
        method2();  
    }  
    private void method2() {  
        method3();  
    }  
    private void method3() {  
        int res = 5/0;  
    }  
}
```

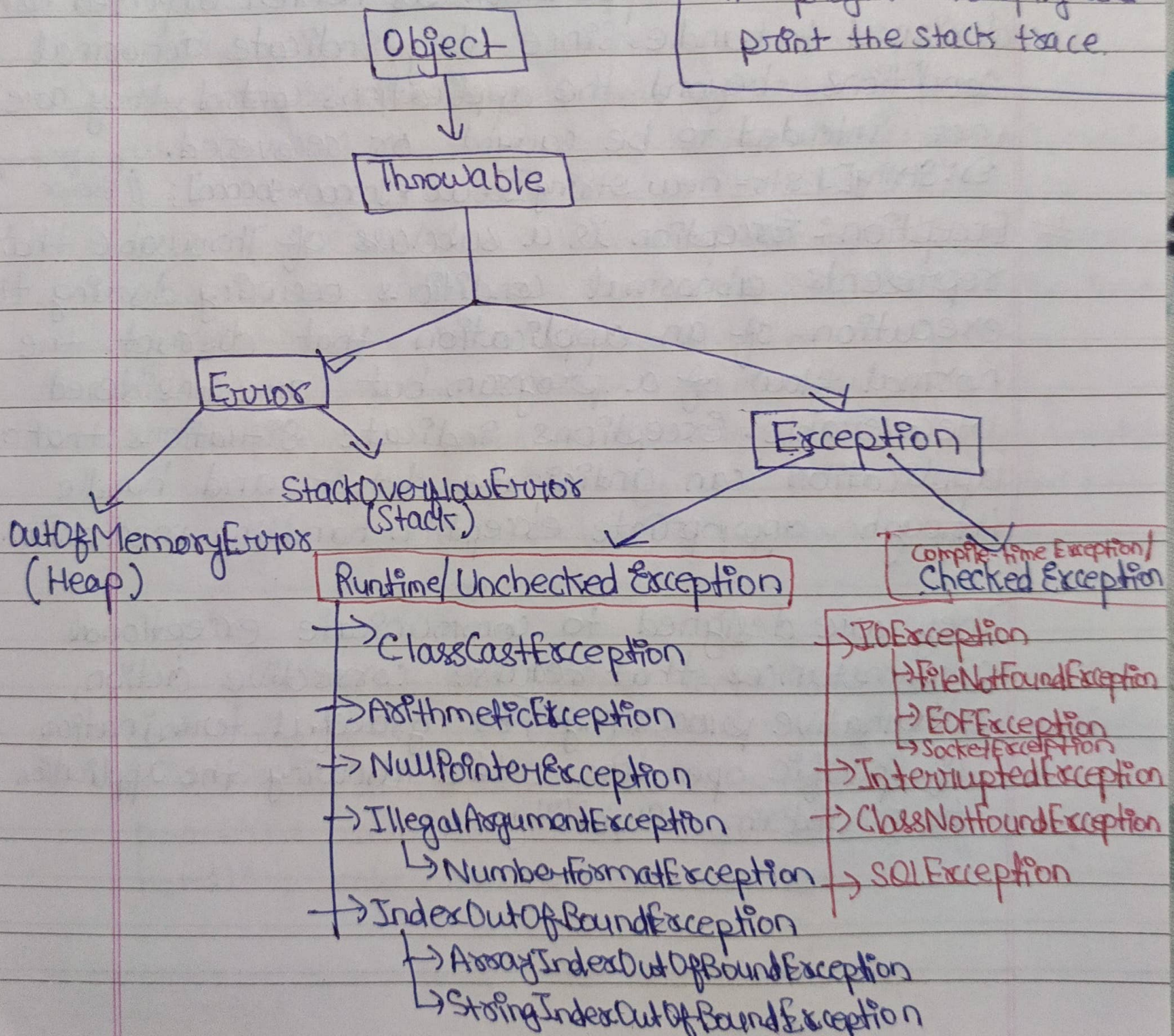
Can method3() handle the excp'n

Exception comes here in method3()



Runtime System will terminate the program abruptly and print the stack trace.

Highlevel Exception Hierarchy



Error: Error is a subclass of Throwable that represent serious unrecoverable problems occurring at the Java Virtual Machine (JVM) level or in the runtime environment.

These conditions indicate failures in the execution environment rather than flaws in normal business logic.

Errors generally arise due to resource exhaustion, JVM internal failures, or fundamental system-level issues that an application is neither expected nor designed to handle. Since they indicate abnormal conditions beyond the application's control, they are not intended to be caught or recovered.

Ex: `String[] str = new String[100000 * 100000 * 100000];` // ^{out of memory} Error

Exception: Exception is a subclass of Throwable that represents abnormal conditions occurring during the execution of an application that disrupt the normal flow of a program but are considered recoverable. Exceptions indicate situations that an application can anticipate, detect, and handle through appropriate exception handling mechanisms.

They are designed to communicate exceptional circumstances that require corrective action, alternative processing, or graceful termination of specific operation while allowing the application to maintain stability.

Checked Exception: checked exception is an exception type that is verified by the Java compiler at compile time, ensuring that any code capable of throwing such an exception either handles it explicitly or declares it for propagation. The compiler enforces this contract because checked exceptions represent recoverable conditions that are reasonably expected during normal program execution and therefore requires deliberate handling by the developer.

This compile time verification promotes reliability by ensuring that potential failure scenarios are acknowledged and addressed before the program is executed.

Unchecked Exceptions: Unchecked Exception is an exception that is not verified by java compiler at compile time and therefore is not required to be explicitly handled or declared. These exceptions typically indicate programming defects, logical errors, invalid assumptions, or improper use of APIs that should ideally be prevented through correct application design rather than runtime recovery.

Responsibility for avoiding unchecked exceptions lies primarily with the developer through proper validation, correct logic and adherence to programming best practices, rather than through mandatory exception handling enforced by the compiler.

Unchecked / Runtime Exception : These exceptions which occurs during runtime and compiler not forcing us to handle them.

Ex:

```
public class Main {
    psum() {
        method1();
    }

    public static void method1() {
        throw new ArithmeticException();
    }
}
```

Types of Unchecked Exception with examples:

i) ClassCastException :

Ex:

```
public class Main {
    psum() {
        Object val = 8; // 8 is of Integer type
        Sout((String) val); // typecasting int to String
    }
}
```

Exception Type/Name

ErrorMessage: Exception in thread "main" java.lang.ClassCastException :
java.lang.Integer cannot be cast to java.lang.String
↳ exception message.

ii) ArithmeticException

```
class Main {
    psum() {
        int val = 5/0;
        Sout(val);
    }
}
```

ErrorMessage: Exception in thread "main" java.lang.ArithmeticException :
/ by zero
↳ exception name

iii) IndexOutOfBoundsException:

i) ArrayIndexOutOfBoundsException:

Ex: class Main {

psvm() {

~~String msg = "hi";~~ int[] arr = {1, 2};

out.println(arr[3]); //

}

}

Error Message: java.lang.~~ArrayIndexOutOfBoundsException~~ ^{ArrayIndexOutOfBoundsException} : 3
↑
name message

ii) StringIndexOutOfBoundsException:

Ex: class Main {

psvm() {

String str = "hello";

out.println(str.charAt(10));

}

}

Error Message: java.lang.StringIndexOutOfBoundsException : // name
String index out of range: 10 // message

iv) NullPointerException:

Ex: public class Main {

psvm() {

String str = null;

out.println(str.charAt(0));

}

}

Error Message: Exception in thread "main" java.lang.NullPointerException
Name only.

V) IllegalArgumentException:

```
Ex: public class Main {
    psvm() {
        int val = Integer.parseInt("abc");
    }
}
```

Error Message: Exception in thread "main" java.lang.NumberFormatException: for input string: "abc"

Checked / Compile time Exception:

→ Compiler verifies them during the compile time of the code and if not handled properly, code compilation will fail.

```
Ex: public class Main {
    psvm() {
        method1();
    }

    public static void method1() {
        throw new ClassNotFoundException();
    }
}
```

Error Message: Main.java:9: Unreported exception ClassNotFoundException; must be caught or declared to be thrown

```
throw new ClassNotFoundException();
```

// Lets try to handle the Exception using "throws";

```

→ public class Main {
    psum() { // caller handles it again program terminates
        method1();
    }

    public static void method1() throws ClassNotFoundException {
        throw new ClassNotFoundException();
    }
}

```

"throws" tells that, this method MIGHT throw this Exception (or might not), so pls caller, you handle it appropriately.

// Lets try to handle the Exception using try/catch block

Ex →

```

public class Main {
    psum() {
        method1();
    }

    public static void method1() {
        try {
            throw new ClassNotFoundException();
        } catch (ClassNotFoundException exceptionObj) {
            // handle this exception scenario like logging
            exceptionObj.printStackTrace();
        }
    }
}

```

Lets talk about now, how to handle the exception.

Using: try, catch, finally, throw, throws.

1. Try/catch :

- Try block specify the code which can throw exception.
- Try block is followed either by catch block or finally block.
- Catch block is used to catch all the exception which can be thrown in the try block.
- Multiple catch block can be used.

Ex: public class Main {

psvmc {

try {

method1("dummy");

? catch (ClassNotFoundException e1) {

// handle it

? catch (InterruptedException e2) {

// handle it

? catch (FileNotFoundException e3) {

// catch block, can only ^{catch} handle excepⁿ, thrown by try block

? catch (Exception e) {

// handle it

}

}

}

public static void method1(String name) throws

ClassNotFoundException, InterruptedException {

if (name.equals("dummy")) {

throw new ClassNotFoundException();

};

throw new InterruptedException();

;

;

;

Specific type
exception
caught only

generic expn
caught all type
of Exception
always use after all
specific expn block

Catch multiple Exception in One Catch block

Syntax: try {

→ Pipe Symbol ("|")

Specific type excep
{ catch (ExceptionType1 | ExceptionType2 | ... e) {
// handle it
e.printStackTrace();
}
generic excep block
{ catch (Exception e) {
// handle all uncatched excep here
}
}

2. Try/Catch/finally or try/finally block

- finally block can be use after try or after catch block.
- finally block will always get executed either if you just return from try block or from catch block.
- At most, we can add only 1 finally block.
- Mostly used for closing the object, adding logs etc.
- If JVM related issues like out of memory, System shut down or our process is forcefully killed. Then finally block do not get executed. cases are (StackOverflow or OutOfMemory) error time.

Syntax: try/catch/finally
try {

{ catch () {

{ finally {

// do something here
// close resources

}

Syntax try/finally
try {

{ finally {

// do something
}

Note: if any excep come in try block it will throws the excep to caller class and caller will handle it.

3. Throw :

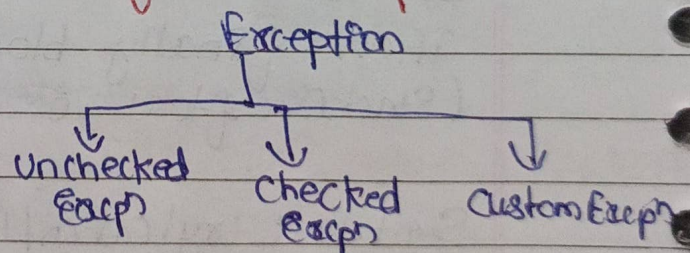
- It is used to throw a new exception or
- To re-throw the exception.

Ex: of re-throw the exception

```
public class Main {
    psvm() throws ClassNotFoundException {
        try {
            method1();
        } catch (ClassNotFoundException e) {
            throw e; // rethrowing excepobj again
        }
    }

    public static void method1() throws ClassNotFoundException {
        throw new ClassNotFoundException();
    }
}
```

creating custom or user defined Exception class



Ex:

```
public class MyCustomException extends Exception {
    // constructor
    MyCustomException(String mess) {
        super(mess);
    }
}
```

```

public class Main {
    psvm() {
        try {
            method1();
        } catch (MyCustomException e) {
            // handle it & out (e.getMessage()); // Issue arise
        }
    }
}
```

27

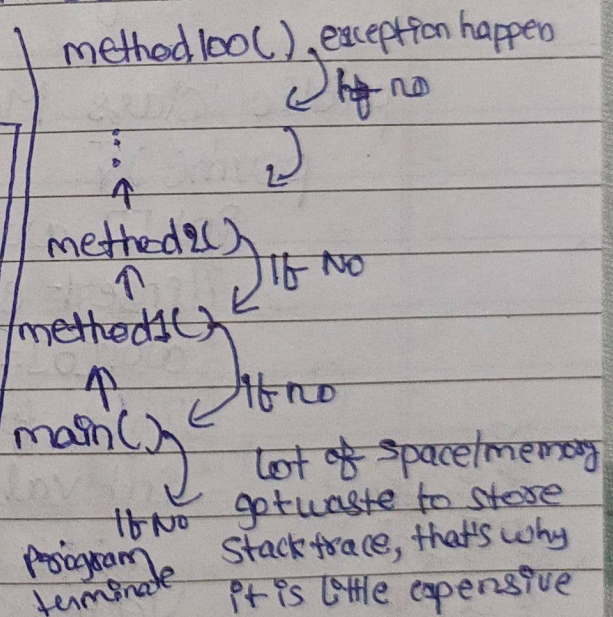
- It makes our code clean by separating the error handling code from regular code.

- It allows program to recover from the errors.
- It allow us to add more information, which support debugging.
- Improves security, by hiding the sensitive info

Note: But remember, exception handling is little expensive, if stack trace is huge and it is not handled or handle at parent class.

9

g



Try to avoid using exception handling, if you can. like in 2nd example.